



Tienda Online

TRABAJO FINAL DEL MÓDULO DE PROYECTO.

Antonio Sanz Pans

Lucas Ezequiel Juncos Luque

Daniel Jordá Conde

Profesor: Albert Llabres Darder

Desarrollo de aplicaciones multiplataforma

Curso 2025-2026

ÍNDICE

INTRODUCCIÓN	3
MOTIVACIONES Y CONTEXTO DEL PROYECTO.....	3
DESCRIPCIÓN DEL PROYECTO	3
FUNCIONALIDADES PRINCIPALES.....	3
STACK TECNOLÓGICO	4
OBJETIVOS	5
OBJETIVOS GENERALES.....	5
OBJETIVOS ESPECÍFICOS.....	5
ENFOQUE Y METODOLOGÍA	6
ENFOQUE DE DESARROLLO.....	6
METODOLOGÍA DE TRABAJO COLABORATIVO	6
METODOLOGÍA TÉCNICA.....	6
ANÁLISIS DE RIESGOS	7
PLANIFICACIÓN TEMPORAL	8
HITOS Y DISTRIBUCIÓN DEL TIEMPO	8
DIAGRAMA DE GANTT.....	9
ANÁLISIS DE REQUISITOS Y CASOS DE USO	10
1. REQUISITOS INICIALES	10
1.1 Requisitos Funcionales	10
1.2 Requisitos No Funcionales.....	10
1.3 Requisitos Técnicos.....	11
2. CASOS DE USO	13
2.1 Diagrama de Casos de Uso	13
2.2 Descripción de Casos de Uso.....	14
2.3. Relaciones entre casos de uso.....	21

Introducción

Motivaciones y Contexto del Proyecto

La elección de este tema responde a la necesidad de consolidar una comprensión rigurosa y aplicada del desarrollo de aplicaciones web modernas. En el contexto actual, donde la transformación digital constituye un eje estratégico para cualquier organización, resulta imprescindible que el profesional del ámbito informático adquiera experiencia real en el desarrollo integral de aplicaciones comerciales desde una perspectiva full-stack.

Se ha seleccionado el desarrollo de una tienda online como caso de estudio por tratarse de un entorno realista y técnicamente exigente, que requiere la integración estructurada de múltiples tecnologías y principios de ingeniería de software. Este planteamiento permite abordar de manera simultánea aspectos técnicos fundamentales —como la definición de la arquitectura, el diseño y gestión de bases de datos o el desarrollo de interfaces de usuario— junto con dimensiones organizativas clave, entre ellas el trabajo colaborativo, el control de versiones y la planificación y seguimiento del proyecto.

Asimismo, el desarrollo de este proyecto facilita la aplicación práctica y cohesionada de los conocimientos adquiridos a lo largo del ciclo formativo, situándolos en un escenario que reproduce dinámicas propias de un entorno profesional real. El trabajo en equipo aporta, además, un valor formativo añadido, al favorecer el desarrollo de competencias transversales como la coordinación, la comunicación efectiva y la resolución colaborativa de problemas, esenciales en el ejercicio profesional del desarrollo de software.

Descripción del Proyecto

El presente trabajo de fin de curso consiste en el desarrollo de una aplicación web que simula el funcionamiento de una tienda online completa. El proyecto abarcará todas las capas fundamentales de una aplicación actual: back-end, front-end y base de datos, garantizando una visión global e integrada del proceso de desarrollo de software moderno.

Funcionalidades Principales

La aplicación incluirá las siguientes características:

Gestión de productos: Catálogo de productos organizados por categorías, con visualización en listado y vistas de detalle ampliadas

Sistema de autenticación: Login persistente

Carrito de compras: Funcionalidad para almacenar productos seleccionados durante la navegación

Simulación de pasarela de pago: Flujo de compra realista sin transacciones monetarias reales

Gestión de usuarios y pedidos: Almacenamiento persistente de información de clientes y su historial de compras

Stack Tecnológico

Back-end: Node.js con JavaScript, responsable de la lógica de negocio, gestión de usuarios y comunicación con la base de datos

Front-end: Web Components reutilizables, proporcionando modularidad, mantenimiento y escalabilidad

Base de datos: Sistema relacional SQL para la gestión de productos, usuarios y pedidos

Control de versiones: GitHub con flujo de trabajo basado en ramas y Pull Requests

Gestión de tareas: Tablero Kanban integrado en GitHub Projects

Objetivos

Objetivos Generales

- Desarrollar una aplicación web funcional que implemente un caso de uso real de comercio electrónico.
- Adquirir experiencia práctica en el desarrollo full-stack utilizando tecnologías actuales.
- Comprender y aplicar metodologías y herramientas utilizadas en entornos profesionales.
- Consolidar la capacidad de trabajo colaborativo en equipo mediante herramientas de desarrollo estándar.

Objetivos Específicos

- Dominar la creación de arquitecturas backend escalables con Node.js y gestión de bases de datos SQL.
- Desarrollar interfaces de usuario modulares y mantenibles utilizando Web Components.
- Implementar sistemas de autenticación y sesiones de usuario seguros y persistentes.
- Aplicar patrones de desarrollo colaborativo: ramas, Pull Requests y revisión de código.
- Implementar la totalidad del flujo de compra, incluyendo carrito y simulación de pago.
- Optimizar la experiencia de usuario mediante una navegación intuitiva y un diseño coherente.
- Documentar adecuadamente el código y la arquitectura del sistema.

Enfoque y Metodología

Enfoque de Desarrollo

Se adoptará un enfoque ágil e iterativo, dividiendo el proyecto en ciclos de desarrollo cortos (sprints) que permitan la entrega incremental de funcionalidades. Cada sprint se alineará con las entregas programadas en el entorno académico, garantizando una planificación coherente con el calendario establecido.

Esta organización permitirá disponer de mecanismos de retroalimentación temprana, facilitando la detección y corrección de posibles desviaciones, así como la adaptación progresiva a cambios o ajustes en los requisitos definidos. De este modo, se favorece un proceso de desarrollo más controlado, flexible y orientado a la mejora continua.

Metodología de Trabajo Colaborativo

Control de versiones: Utilización de GitHub con un flujo de ramas (Git Flow) donde la rama “master” permanece estable y los cambios se integran mediante Pull Requests revisadas.

Gestión de tareas: Tablero Kanban en GitHub Projects para visualizar el estado de las tareas (Backlog, Ready, In progress, Done, Discarded).

Comunicación: Reuniones periódicas de sincronización y uso de canales de comunicación asincrónica.

Revisión de código: Cada Pull Request será revisada al menos por un compañero antes de su integración.

Metodología Técnica

Desarrollo modular: Implementación de componentes independientes y reutilizables en el front-end.

Separación de capas: Distinción clara entre lógica de presentación, lógica de negocio y acceso a datos

Testing: Implementación de pruebas unitarias para las funcionalidades críticas.

Documentación: Generación de documentación técnica clara y accesible para el código y la arquitectura.

Análisis de Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Limitada experiencia inicial con las tecnologías empleadas	Alta	Alto	Establecer sesiones de formación previas, consultar documentación oficial, aprender durante el desarrollo inicial
Descoordinación del equipo por trabajar completamente online	Alta	Alto	Establecer reuniones regulares, definir canales de comunicación claros, crear documentación compartida
Incompatibilidades de horarios por responsabilidades laborales y familiares	Media	Medio	Flexibilidad en la planificación, trabajo asincrónico bien documentado, alertas anticipadas de cambios de disponibilidad
Conflictos de versión en el control de versiones	Media	Medio	Buenas prácticas de Git, ramas bien organizadas, comunicación antes de cambios importantes
Alcance excesivo del proyecto	Media	Medio	Definición clara de funcionalidades mínimas (MVP), priorización de tareas, diferenciación de mejoras opcionales
Retrasos en la integración de componentes	Media	Medio	Interfaces bien definidas previamente, testing durante desarrollo, integraciones frecuentes
Problemas de rendimiento en la aplicación	Baja	Alto	Análisis de rendimiento temprano, optimización iterativa, monitoreo durante la fase de testing

Planificación Temporal

El proyecto se estructurará en las siguientes fases:

Fase	Duración	Hito
Fase 1: Planificación y Diseño	14	Arquitectura definida, base de datos diseñada, equipo configurado
Fase 2: Setup Inicial	7	Entorno de desarrollo configurado, repositorio preparado, estructura de proyecto creada
Fase 3: Desarrollo Backend (MVP)	21	Diseño de API REST, modelos de datos y lógica de autenticación
Fase 4: Desarrollo Frontend (MVP)	21	Componentes principales, navegación, integración con API
Fase 5: Integración y Testing	14	Sistema integrado, testing completo, corrección de errores críticos
Fase 6: Refinamiento y Mejoras	14	Optimización, mejoras opcionales, corrección de errores documentación final
Fase 7: Revisión Final y Entrega	12	Testing final, documentación completa, presentación preparada

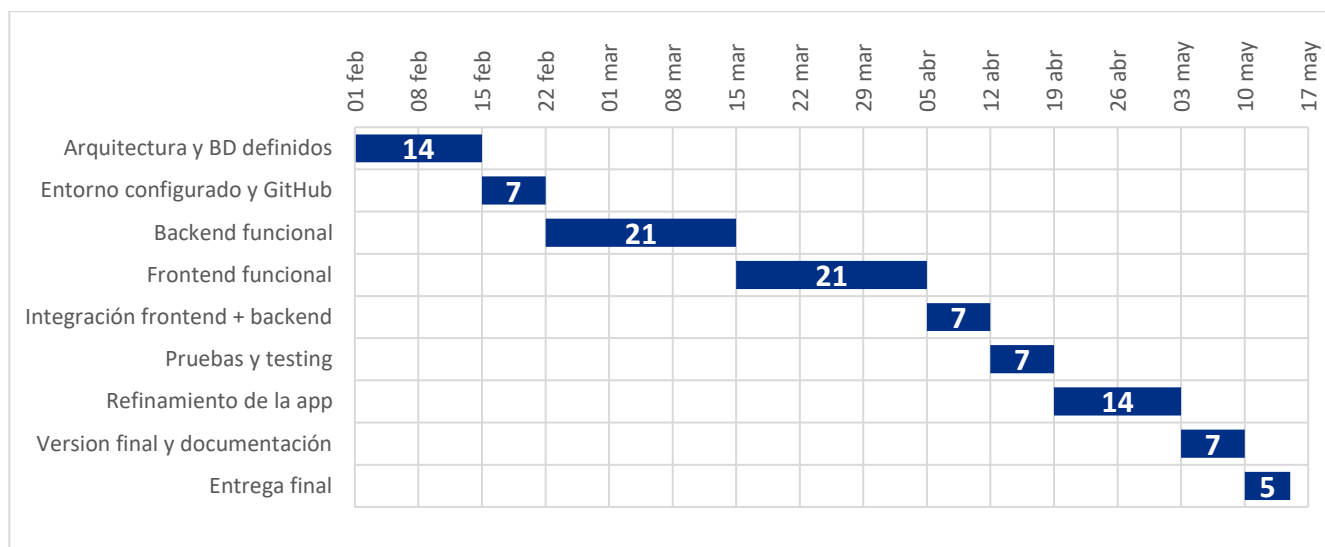
Hitos y Distribución del Tiempo

Los hitos tendrán asignados unas fechas aproximadas que podrían variar según se desarrolle el proyecto:

Hito	Inicio	Duración	Fin
Arquitectura y BD definidos	01/02/2026	14	14/02/2026
Entorno configurado y GitHub	15/02/2026	7	21/02/2026
Backend funcional	22/02/2026	21	14/03/2026
Frontend funcional	15/03/2026	21	04/04/2026
Integración frontend + backend	05/04/2026	7	11/04/2026
Pruebas y testing	12/04/2026	7	18/04/2026
Refinamiento de la app	19/04/2026	14	02/05/2026
Versión final y documentación	03/05/2026	7	09/05/2026
Entrega final	10/05/2026	5	14/05/2026
		103	

Fechas aproximadas

Diagrama de Gantt



ANÁLISIS DE REQUISITOS Y CASOS DE USO

1. REQUISITOS INICIALES

1.1 Requisitos Funcionales

Los requisitos funcionales definen las funcionalidades específicas que debe cumplir el sistema:

1.1.1 Gestión de Usuarios

- **RF-001:** El sistema debe permitir el registro de nuevos usuarios con validación de datos.
- **RF-002:** El sistema debe permitir la autenticación de usuarios mediante credenciales.
- **RF-003:** El sistema debe permitir la visualización y edición del perfil de usuario.
- **RF-004:** El sistema debe permitir la recuperación de contraseña mediante correo electrónico.
- **RF-005:** El sistema debe permitir la gestión de roles y permisos de usuarios.

1.1.2 Gestión de Datos

- **RF-006:** El sistema debe permitir crear, leer, actualizar y eliminar registros (CRUD).
- **RF-007:** El sistema debe permitir la búsqueda avanzada de registros con múltiples filtros.

1.1.3 Interfaz de Usuario

- **RF-008:** El sistema debe proporcionar una interfaz responsiva compatible con dispositivos móviles y de escritorio.
- **RF-009:** El sistema debe incluir un panel de control (dashboard) con métricas principales.
- **RF-010:** El sistema debe permitir la personalización de la interfaz según preferencias del usuario.
- **RF-011:** El sistema debe proporcionar mensajes claros de retroalimentación en todas las operaciones.

1.1.4 Notificaciones

- **RF-012:** El sistema debe generar notificaciones en tiempo real para eventos importantes.

1.1.5 Seguridad

- **RF-013:** El sistema debe validar todos los datos de entrada contra inyecciones.
- **RF-014:** El sistema debe cifrar datos sensibles tanto en tránsito como en reposo.

1.2 Requisitos No Funcionales

Los requisitos no funcionales definen las características de calidad del sistema:

1.2.1 Disponibilidad y Confiabilidad

- **RNF-001:** El sistema debe estar disponible durante el desarrollo y las pruebas en entorno local. No se establecen compromisos de disponibilidad en producción al tratarse de un proyecto académico.

1.2.2 Rendimiento

- **RNF-002:** El tiempo de respuesta de cualquier operación no debe exceder 3 segundos.
- **RNF-003:** La aplicación debe cargar en menos de 2 segundos.
- **RNF-004:** Las consultas a base de datos deben completarse en menos de 1 segundo.

1.2.3 Escalabilidad

- **RNF-005:** El sistema debe diseñarse de forma modular para facilitar futuras ampliaciones, sin requisitos de escala definidos en esta fase académica.
- **RNF-006:** La arquitectura debe separar claramente las capas de presentación, lógica de negocio y acceso a datos, facilitando el mantenimiento y la comprensión del código.
- **RNF-007:** El sistema debe gestionar al menos 10GB de datos sin problemas de rendimiento.

1.2.4 Seguridad

- **RNF-008:** Las contraseñas deben cumplir una política de complejidad mínima.
- **RNF-009:** Todas las contraseñas deben de almacenarse encriptadas.

1.2.5 Usabilidad

- **RNF-010:** La interfaz debe ser intuitiva sin necesidad de formación adicional.
- **RNF-011:** El sistema debe estar disponible en múltiples idiomas (mínimo inglés y español).
- **RNF-012:** El tiempo de aprendizaje para nuevos usuarios no debe exceder 30 minutos.

1.2.6 Mantenibilidad

- **RNF-013:** El código debe seguir estándares de buenas prácticas y convenciones establecidas.
- **RNF-014:** La documentación debe mantenerse actualizada con el código.
- **RNF-015:** La arquitectura debe separar claramente las capas de presentación, lógica de negocio y acceso a datos, facilitando el mantenimiento y la comprensión del código.

1.2.7 Compatibilidad

- **RNF-016:** El sistema debe ser compatible con los últimos navegadores (Chrome, Firefox, Safari, Edge).
- **RNF-017:** El sistema debe funcionar en dispositivos iOS y Android.

1.3 Requisitos Técnicos

Los requisitos técnicos especifican la tecnología y arquitectura a utilizar:

1.3.1 Frontend

- **RT-001:** Desarrollo con JavaScript/React como framework principal.
- **RT-002:** Uso de herramientas de construcción (Webpack, Vite) para optimización.
- **RT-003:** Implementación de Context API para gestión de estado.
- **RT-004:** Uso de librerías de UI (Bootstrap) para consistencia visual.
- **RT-005:** Implementación de CSS responsive y Mobile-First.

1.3.2 Backend

- **RT-006:** Desarrollo con Node.js como entorno de ejecución.
- **RT-007:** Uso de Express.js como framework web.
- **RT-008:** Implementación de APIs RESTful con estándares OpenAPI/Swagger.

1.3.3 Base de Datos

- **RT-009:** SQL como base de datos principal para datos estructurados.

1.3.4 Testing y Calidad

- **RT-010:** Tests unitarios con Jest o Mocha.
- **RT-011:** Tests de integración para APIs.
- **RT-012:** Tests end-to-end con Cypress o Selenium.
- **RT-013:** Cobertura de código mínima del 80%.
- **RT-014:** Linting y formateado automático con ESLint y Prettier.

2. CASOS DE USO

2.1 Diagrama de Casos de Uso



2.2 Descripción de Casos de Uso

UC-001: Registro de Usuario

Actor Principal: Invitado

Descripción: Un usuario sin cuenta crea una nueva cuenta en la tienda.

Precondiciones:

- El usuario no debe estar autenticado.
- El formulario de registro debe estar disponible.

Flujo Principal:

1. El invitado accede a la página de registro.
2. El sistema muestra el formulario con los campos requeridos (nombre, email, contraseña).
3. El invitado completa y envía el formulario.
4. El sistema valida los datos introducidos.
5. El sistema crea la cuenta y almacena los datos en la base de datos.
6. El sistema redirige al usuario a la página principal como cliente autenticado.
7. El caso de uso finaliza satisfactoriamente.

Flujos Alternativos:

- **3a. Datos inválidos:** El sistema muestra mensajes de error específicos por campo.
- **3b. Email ya registrado:** El sistema informa de que ya existe una cuenta con ese email y sugiere iniciar sesión.

Postcondiciones:

- Se crea una nueva cuenta de cliente en el sistema.
- El usuario queda autenticado automáticamente.

UC-002: Autenticación de Usuario

Actor Principal: Invitado

Descripción: Un usuario con cuenta existente inicia sesión en la tienda.

Precondiciones:

- El usuario debe tener una cuenta registrada.

Flujo Principal:

1. El invitado accede a la página de login.
2. El invitado introduce su email y contraseña.
3. El sistema valida las credenciales.
4. El sistema genera una sesión para el usuario.
5. El sistema redirige al cliente a la página principal.
6. El caso de uso finaliza satisfactoriamente.

Flujos Alternativos:

- **2a. Credenciales incorrectas:** El sistema muestra un mensaje de error genérico y permite reintentar.
- **2b. Cuenta no existe:** El sistema sugiere al usuario registrarse.

Postcondiciones:

- El usuario está autenticado y dispone de una sesión activa.

UC-003: Consultar Catálogo de Productos

Actor Principal: Invitado, Cliente

Descripción: El usuario navega por el catálogo de productos disponibles.

Precondiciones:

- Deben existir productos publicados en la base de datos.

Flujo Principal:

1. El usuario accede a la sección del catálogo.
2. El sistema recupera los productos de la base de datos.
3. El sistema muestra los productos en formato listado con imagen, nombre y precio.
4. El usuario puede hacer clic en un producto para ver su detalle.
5. El sistema muestra la página de detalle con descripción, precio y disponibilidad.
6. El caso de uso finaliza.

Flujos Alternativos:

- **2a. Sin productos:** El sistema muestra un mensaje informativo indicando que no hay productos disponibles.
- **4a. Error de carga:** El sistema muestra un mensaje de error e invita a recargar la página.

Postcondiciones:

- El usuario ha podido visualizar los productos disponibles.

UC-004: Búsqueda y Filtrado de Productos

Actor Principal: Invitado, Cliente

Descripción: El usuario busca productos por nombre o los filtra por categoría.

Precondiciones:

- Deben existir productos en el sistema.

Flujo Principal:

1. El usuario introduce un término en el buscador o selecciona una categoría.
2. El sistema realiza la consulta en la base de datos.
3. El sistema muestra los productos que coinciden con el criterio.
4. El usuario puede combinar búsqueda y filtro simultáneamente.
5. El caso de uso finaliza.

Flujos Alternativos:

- **3a. Sin resultados:** El sistema informa de que no hay productos que coincidan con la búsqueda.
- **1a. Búsqueda vacía:** El sistema muestra todos los productos disponibles.

Postcondiciones:

- Se muestran únicamente los productos que cumplen los criterios indicados.

UC-005: Gestionar Carrito de Compras

Actor Principal: Cliente

Descripción: El cliente añade, modifica o elimina productos de su carrito de compras.

Precondiciones:

- El cliente debe estar autenticado.
- Deben existir productos disponibles.

Flujo Principal:

1. El cliente accede al detalle de un producto.
2. El cliente pulsa "Añadir al carrito".
3. El sistema añade el producto al carrito y actualiza el contador.
4. El cliente accede a la vista del carrito.
5. El sistema muestra los productos añadidos con cantidades y precio total.
6. El cliente puede modificar cantidades o eliminar productos.
7. El sistema actualiza el total en tiempo real.
8. El caso de uso finaliza.

Flujos Alternativos:

- **2a. Invitado intenta añadir al carrito:** El sistema redirige al login e indica que es necesario iniciar sesión.
- **6a. Carrito vacío tras eliminar:** El sistema muestra un mensaje indicando que el carrito está vacío.

Postcondiciones:

- El carrito refleja correctamente los productos y cantidades seleccionados.

UC-006: Realizar Pedido (Simulación de Pago)

Actor Principal: Cliente

Descripción: El cliente finaliza la compra introduciendo sus datos y confirmando el pedido mediante una pasarela de pago simulada.

Precondiciones:

- El cliente debe estar autenticado.
- El carrito debe contener al menos un producto.

Flujo Principal:

1. El cliente accede al carrito y pulsa "Finalizar compra".
2. El sistema muestra el formulario de datos de envío.
3. El cliente introduce la dirección de entrega.
4. El sistema muestra el resumen del pedido con el total.
5. El cliente accede a la pantalla de pago simulado.
6. El cliente introduce datos de tarjeta ficticios.
7. El sistema simula la validación del pago.
8. El sistema registra el pedido en la base de datos con estado "confirmado".
9. El sistema muestra una pantalla de confirmación con el número de pedido.
10. El caso de uso finaliza satisfactoriamente.

Flujos Alternativos:

- **7a. Pago rechazado (simulado):** El sistema muestra un mensaje de pago no aceptado y permite reintentar.
- **3a. Datos de envío incompletos:** El sistema indica los campos obligatorios que faltan.

Postcondiciones:

- El pedido queda registrado en la base de datos con estado "confirmado".
- El carrito queda vaciado.

UC-007: Gestionar Productos (Administrador)

Actor Principal: Administrador

Descripción: El administrador crea, edita o elimina productos del catálogo.

Precondiciones:

- El administrador debe estar autenticado con rol de administrador.

Flujo Principal:

1. El administrador accede al panel de administración.
2. El administrador selecciona la sección de productos.
3. El sistema muestra el listado de productos existentes.
4. El administrador puede crear un nuevo producto, editar uno existente o eliminarlo.
5. El sistema valida los datos introducidos.
6. El sistema guarda los cambios en la base de datos.
7. El sistema muestra confirmación de la operación.
8. El caso de uso finaliza satisfactoriamente.

Flujos Alternativos:

- **5a. Datos inválidos:** El sistema muestra los errores de validación correspondientes.
- **4a. Eliminar producto con pedidos asociados:** El sistema advierte al administrador antes de proceder.

Postcondiciones:

- El catálogo de productos queda actualizado con los cambios realizados.

2.3. Relaciones entre casos de uso

Caso de uso	Relación	Relacinado con
UC-001	Alternativa a	UC-002
UC-002	Precede	UC-005, UC-006
UC-003	Precede	UC-004, UC-005
UC-004	Especializacion de	UC-003
UC-005	Precede	UC-006
UC-006	Depende de	UC-002, UC-005
UC-007	Independiente	-